

Análisis numérico

Tema 2. Solución numérica de ecuaciones algebraicas y trascendentes

Ing. Eduardo Flores Rivas

Facultad de Ingeniería
Universidad Nacional Autónoma de México

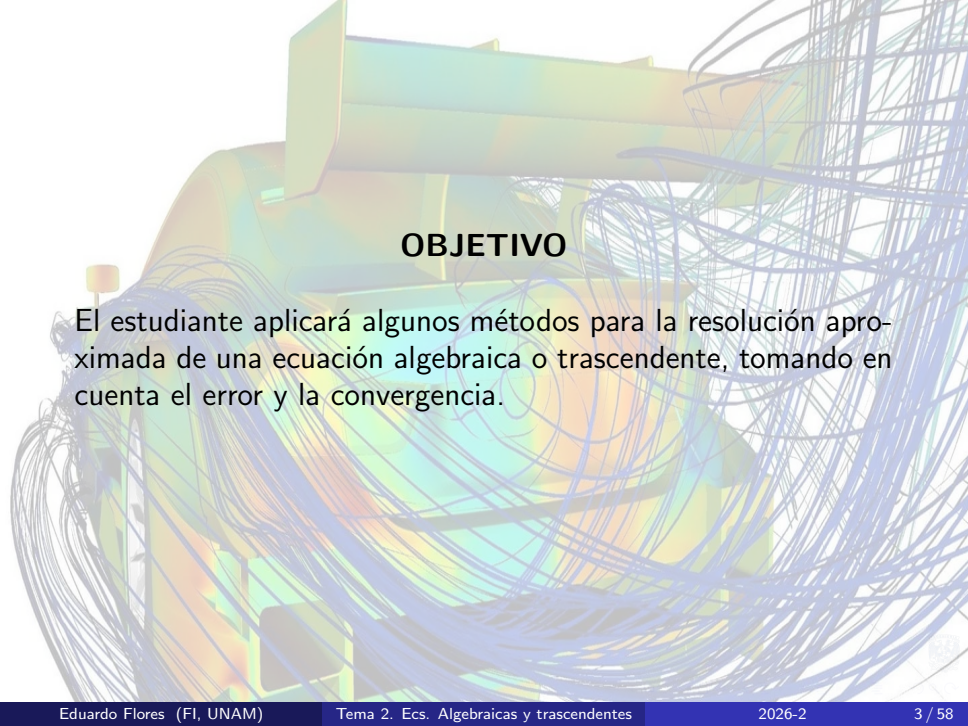
Semestre 2026-2



Contenido

- 1 Objetivo
- 2 Ecuaciones algebraicas y trascendentes
- 3 Métodos cerrados
 - Método de bisección
 - Método de interpolación lineal (regla falsa)
- 4 Métodos abiertos
 - Método de aproximaciones sucesivas
 - Método de Newton-Raphson
- 5 Método de factores cuadráticos
- 6 Contacto
- 7 Bibliografía





OBJETIVO

El estudiante aplicará algunos métodos para la resolución aproximada de una ecuación algebraica o trascendente, tomando en cuenta el error y la convergencia.

Ecuaciones algebraicas: Definición y características

Definición

Una **ecuación algebraica** es una igualdad matemática que involucra una o más variables y donde las operaciones permitidas son:

- Suma, resta, multiplicación, división
- Potenciación con exponentes **racionales** (enteros o fraccionarios)
- Radicales (raíces n -ésimas)

Forma general de ecuación algebraica en una variable:

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = 0, \quad a_n \neq 0$$

donde n es un entero no negativo (grado del polinomio).

Teorema fundamental del álgebra: Toda ecuación algebraica de grado n tiene exactamente n raíces complejas (contando multiplicidades).



Definición

Una **ecuación trascendente** es aquella que **no puede reducirse** a una ecuación algebraica. Involucra funciones trascendentes:

- Exponenciales (e^x , a^x)
- Logarítmicas ($\ln x$, $\log_a x$)
- Trigonómicas ($\sin x$, $\cos x$, $\tan x$)
- Hiperbólicas ($\sinh x$, $\cosh x$)
- Sus inversas y combinaciones

Ejemplos comunes en ingeniería:

- **Exponencial:** $e^{-x} - x = 0$ (ley de enfriamiento de Newton)
- **Trigonómica:** $x - \cos x = 0$ (problemas de oscilación)
- **Logarítmica:** $\ln(x) + x^2 = 5$ (química, pH)
- **Mixta:** $e^{-x} = \sin x$ (circuitos RLC)
- **Implícita:** $x \sin x = 1$ (óptica, difracción)



Algebraico vs trascendental

Diferencias clave:

Ecuaciones Algebraicas	Ecuaciones Trascendentes
Solución por fórmulas hasta 4° grado	Generalmente sin solución analítica cerrada
Número finito de raíces (grado n)	Pueden tener infinitas raíces (sin $x = 0$)
Métodos simbólicos posibles	Requieren métodos numéricos
Ejemplo: $x^2 - 2 = 0 \Rightarrow x = \pm\sqrt{2}$	Ejemplo: $e^x = 2 \Rightarrow x = \ln 2$

Importancia en métodos numéricos: Las ecuaciones trascendentes son el **motivo principal** para desarrollar métodos numéricos como bisección, Newton-Raphson, etc.

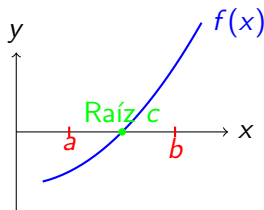


2.1 Métodos cerrados: Introducción

Definición

Los **métodos cerrados** o **métodos de intervalo** son algoritmos que requieren un intervalo inicial $[a, b]$ donde se garantice la existencia de al menos una raíz, mediante el **teorema de valor intermedio**.

Caso típico:



Hipótesis fundamental:

- $f(x)$ continua en $[a, b]$
- $f(a) \cdot f(b) < 0$ (cambia de signo en los extremos)
- $\Rightarrow$ Existe al menos un $c \in (a, b)$ tal que $f(c) = 0$



Propiedades comunes:

- Garantía de convergencia
- Estabilidad numérica
- Convergencia lenta (lineal)
- Requiere intervalo inicial con cambio de signo

Aplicaciones:

- Ecuaciones algebraicas (polinomios)
- Ecuaciones trascendentes (e^x , $\sin x$, $\log x$, etc.)
- Problemas con raíces aisladas

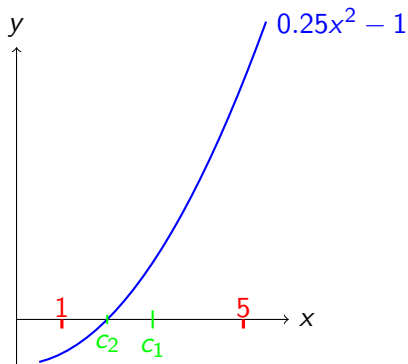


Método de bisección

Idea geométrica

Dividir repetidamente el intervalo a la mitad y seleccionar el subintervalo que contiene la raíz, basándose en el cambio de signo.

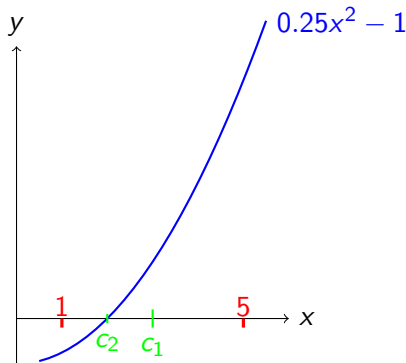
Interpretación geométrica:



Algoritmo paso a paso:

Algoritmo del método de bisección

Interpretación geométrica:



Algoritmo paso a paso:

- ➊ Entrada: $a, b, f(x), \varepsilon_{\text{tol}}, N_{\text{máx}}$
- ➋ Verificar: $f(a) \cdot f(b) < 0$
- ➌ Para $i = 1, 2, \dots, N_{\text{máx}}$:
 - ➊ $c = \frac{a+b}{2}$ (punto medio)
 - ➋ Si $f(c) = 0$ o $E_a < \varepsilon_{\text{tol}}$: Terminar
 - ➌ Si $f(a) \cdot f(c) < 0$: $b = c$ (raíz en $[a, c]$)
 - ➍ Si $f(b) \cdot f(c) < 0$: $a = c$ (raíz en $[c, b]$)
- ➍ Salida: c (aproximación de la raíz)



Ejemplo: Método de bisección

Problema: Encontrar raíz de $f(x) = x^3 - 2x - 5 = 0$ en $[2, 3]$

Datos: $a_0 = 2$, $b_0 = 3$, $\varepsilon_{\text{tol}} = 0.001$

iteración	a	c	b	f(a)	f(c)	f(b)	error
0	2.00000	–	3.00000	-1.00000	–	16.00000	–
1	2.00000	2.50000	3.00000	-1.00000	5.62500	16.00000	1.00000
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮



Solución: Método de bisección

Problema: Encontrar raíz de $f(x) = x^3 - 2x - 5 = 0$ en $[2, 3]$

Datos: $a_0 = 2$, $b_0 = 3$, $\varepsilon_{\text{tol}} = 0.001$

iteración	a	c	b	f(a)	f(c)	f(b)	error
0	2.00000	–	3.00000	-1.00000	–	16.00000	–
1	2.00000	2.50000	3.00000	-1.00000	5.62500	16.00000	1.00000
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
10	2.09375	2.094727	2.095703	-0.008942	0.001954	0.012862	0.000977

Análisis de resultados:

- Raíz aproximada: $c_{10} = 2.0947$
- Error absoluto garantizado: $|E_{10}| \leq 0.00098 < 0.001$
- Iteraciones necesarias:
Teóricas $n \geq \frac{\ln(1/0.001)}{\ln 2} \approx 9.97 \Rightarrow 10$ iteraciones
- Valor real: $x^* \approx 2.09455148$, error real:
 $|2.0947 - 2.094551| \approx 0.000149$



Implementación 1: Método de Bisección

Objetivo: Implementar y comprender el algoritmo del método de bisección para encontrar raíces de ecuaciones.

Actividades:

1 Análisis algorítmico:

- Escribir el pseudocódigo completo del método de bisección
- Realizar el diagrama de flujo correspondiente
- Especificar claramente: entradas, proceso, salidas

2 Implementación en Python:

- Programar la función: `biseccion(f, a, b, tol, max_iter)`
- La función debe realizar las verificaciones adecuadas en los datos de entrada y contar con criterio de detención según la tolerancia especificada y el número máximo de iteraciones.
- Se espera a la salida la raíz aproximada, el número de iteraciones y el error absoluto final.

Entregables (por Classroom):

- Documento PDF con: pseudocódigo, diagrama de flujo, código Python
- Archivo .py con la implementación

Fecha límite: 11 de marzo, 2026.



Tarea 3: Método de Bisección

Problema: Encontrar raíz de $f(x) = \sin(x) = 0$ en $[2, 4]$ usando el método de bisección.

- **Datos:** $a_0 = 2$, $b_0 = 4$, $\varepsilon_{\text{tol}} = 10^{-3}$
- Comparar resultado con valor analítico π
- Generar tabla de iteraciones: n , a_n , c_n , b_n , $f(a_n)$, $f(c_n)$, $f(b_n)$, error

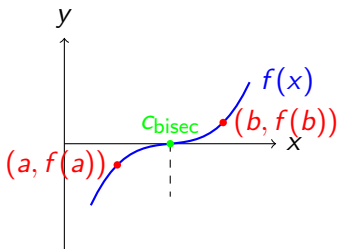


Limitación del método de bisección

Desventaja del método

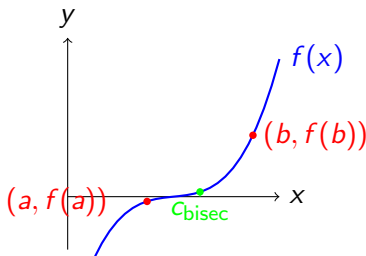
El método de bisección utiliza **solamente el signo** de $f(a)$ y $f(b)$, pero **ignora completamente la magnitud** de estos valores. Esto puede llevar a convergencia más lenta de lo necesario.

Caso 1: Valores equilibrados



Bisección funciona bien:

Caso 2: Valores desequilibrados



Problema: Raíz está más
cerca de a

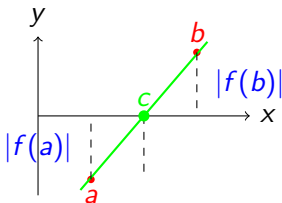


Motivación uso de magnitud: Interpolación

¿Cómo mejorar?

En lugar de usar solo el signo, usar la **magnitud relativa** de $f(a)$ y $f(b)$ para estimar dónde la función cruza el eje x .

Modelo lineal



Derivación de la fórmula

Despejando c :

$$\begin{aligned}(c - a)f(b) &= -(b - c)f(a) \\ c \cdot f(b) - a \cdot f(b) &= -b \cdot f(a) + c \cdot f(a) \\ c(f(b) - f(a)) &= a \cdot f(b) - b \cdot f(a)\end{aligned}$$

Por semejanza de triángulos:

$$\frac{c - a}{b - c} = \frac{|f(a)|}{|f(b)|} = \frac{-f(a)}{f(b)}$$

Fórmula de interpolación lineal

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$



Método de interpolación lineal (regla falsa)

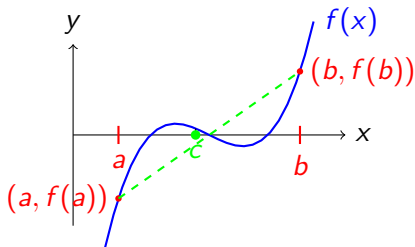
Idea geométrica

En lugar de usar el punto medio, usar la intersección de la recta secante que une $f(a)$ y $f(b)$ con el eje x .

Fórmula de iteración:

$$c = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)} = b - \frac{f(b)(b - a)}{f(b) - f(a)} = a - \frac{f(a)(b - a)}{f(b) - f(a)}$$

Interpretación geométrica:



Algoritmo:

- 1 Mismo que bisección, pero fórmula diferente para c
- 2 Criterio de selección de subintervalo igual
- 3 Normalmente converge más rápido que bisección



Ejemplo: Método de regla falsa

Problema: Encontrar raíz de $f(x) = x^3 - 2x - 5 = 0$ en $[2, 3]$

Datos: $a_0 = 2$, $b_0 = 3$, $\varepsilon_{\text{tol}} = 0.001$

iteracion	a	c	b	f(a)	f(c)	f(b)	error
0	2.000000	NaN	3	-1.000000	NaN	16	NaN
1	2.000000	2.058824	3	-1.000000	-0.390800	16	1.000000
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮



Ejemplo: Método de regla falsa

Problema: Encontrar raíz de $f(x) = x^3 - 2x - 5 = 0$ en $[2, 3]$

Datos: $a_0 = 2$, $b_0 = 3$, $\varepsilon_{\text{tol}} = 0.001$

iteracion	a	c	b	f(a)	f(c)	f(b)	error
0	2.000000	NaN	3	-1.000000	NaN	16	NaN
1	2.000000	2.058824	3	-1.000000	-0.390800	16	1.000000
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
6	2.093884	2.094305	3	-0.007451	-0.002746	16	0.000422



Implementación 2: Método de regla falsa

Objetivo: Implementar y comprender el algoritmo del método de regla falsa para encontrar raíces de ecuaciones.

Actividades:

1 Análisis algorítmico:

- Escribir el pseudocódigo completo del método de regla falsa
- Realizar el diagrama de flujo correspondiente
- Especificar claramente: entradas, proceso, salidas

2 Implementación en Python:

- Programar la función: `regla_falsa(f, a, b, tol, max_iter)`
- La función debe realizar las verificaciones adecuadas en los datos de entrada y contar con criterio de detención según la tolerancia especificada y el número máximo de iteraciones.
- Se espera a la salida la raíz aproximada, el número de iteraciones y el error absoluto final.

Entregables (por Classroom):

- Documento PDF con: pseudocódigo, diagrama de flujo, código Python
- Archivo .py con la implementación

Fecha límite: [Indicar fecha]



Tarea 4: Método de regla falsa

Problema: Encontrar raíz de $f(x) = \sin(x) = 0$ en $[2, 4]$ usando el método de regla falsa.

- **Datos:** $a_0 = 2$, $b_0 = 4$, $\varepsilon_{\text{tol}} = 10^{-3}$
- Comparar resultado con valor analítico π
- Generar tabla de iteraciones: n , a_n , c_n , b_n , $f(a_n)$, $f(c_n)$, $f(b_n)$, error



Métodos abiertos

Los **métodos abiertos** o **métodos de punto fijo** son algoritmos que requieren un valor inicial x_0 a partir del cuál se busca la raíz de la ecuación.

Teorema del punto fijo

Sea $g : [a, b] \rightarrow [a, b]$ una función continua y diferenciable. Si existe una constante $0 \leq k < 1$ tal que:

$$|g'(x)| \leq k < 1 \quad \forall x \in [a, b]$$

entonces:

- 1 Existe un único punto fijo $x^* \in [a, b]$ tal que $x^* = g(x^*)$
- 2 La sucesión $x_{n+1} = g(x_n)$ converge a x^* para cualquier $x_0 \in [a, b]$
- 3 El error satisface: $|x_n - x^*| \leq \frac{k^n}{1-k} |x_1 - x_0|$

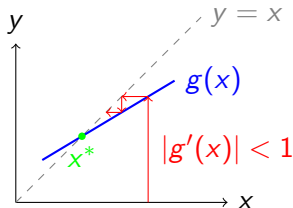


Convergencia

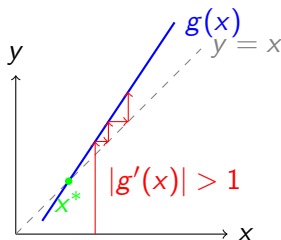
La convergencia depende de $|g'(x^*)|$:

- $|g'(x^*)| < 1$: Convergencia lineal
- $|g'(x^*)| = 0$: Convergencia superlineal (ideal)
- $|g'(x^*)| \geq 1$: Posible divergencia

Convergencia (caso favorable)



Divergencia (caso desfavorable)



Método de aproximaciones sucesivas (punto fijo)

Definición

Dada una ecuación $f(x) = 0$, se lleva a la forma equivalente $x = g(x)$. El método genera una sucesión: $x_{n+1} = g(x_n)$, $n = 0, 1, 2, \dots$ donde x_0 es una aproximación inicial y $g(x)$ es la **función de iteración**.

Algoritmo básico:

- 1 Transformar $f(x) = 0$ a $x = g(x)$
- 2 Elegir aproximación inicial x_0
- 3 Para $n = 0, 1, 2, \dots$
 - Calcular $x_{n+1} = g(x_n)$
 - Verificar convergencia:
 $|x_{n+1} - x_n| < \varepsilon$
 - Si converge, x_{n+1} es raíz aproximada

Criterio de parada:

$$|x_{n+1} - x_n| < \varepsilon \quad \text{o} \quad |f(x_n)| < \delta$$

Observaciones de $g(x)$:

- La convergencia depende de $g(x)$
- Distintas funciones $g(x)$ dan distintos resultados

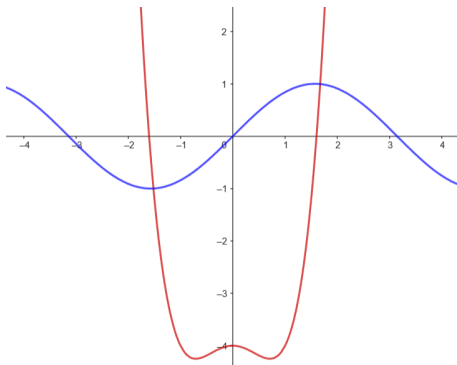


Solución alternativa: Gráfica de dos curvas

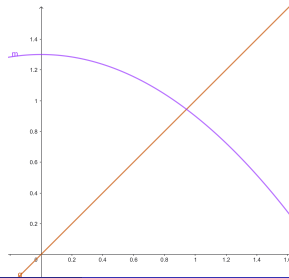
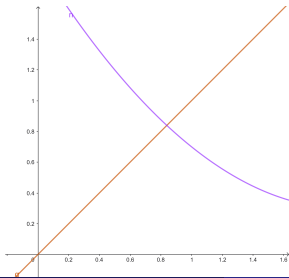
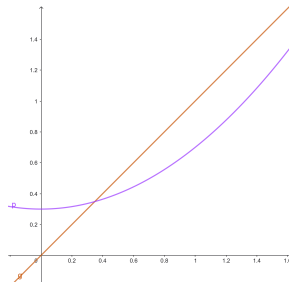
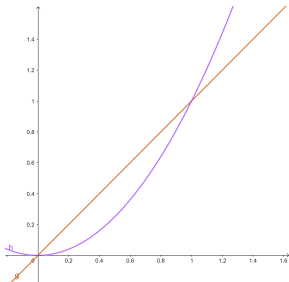
Otra forma de trabajar con el **método de iteraciones sucesivas** consiste en separar la ecuación $f(x) = 0$ a resolver en dos funciones de manera

$$f_1(x) = f_2(x)$$

Si se grafican las funciones $y_1 = f_1(x)$ y $y_2 = f_2(x)$, los puntos de intersección representa la solución del sistema.



¿Qué ocurre en los siguientes casos? ¿Converge?



Ejemplo: Transformación de $f(x) = 0$ a $x = g(x)$

Problema: Proponer función $g(x)$ para la función $f(x) = x^2 - x - 2 = 0$ con raíces: $x = -1, 2$

Posibles transformaciones:

- $g_1(x) = x^2 - 2$

$$x^2 - x - 2 = 0 \Rightarrow x = x^2 - 2$$

- $g_2(x) = \sqrt{x+2}$ (válida para $x \geq -2$)

$$x^2 - x - 2 = 0 \Rightarrow x^2 = x + 2 \Rightarrow x = \sqrt{x+2}$$

- $g_3(x) = \frac{2}{x-1}$ ($x \neq 1$)

$$x^2 - x - 2 = 0 \Rightarrow x(x-1) = 2 \Rightarrow x = \frac{2}{x-1}$$



Análisis de convergencia de $g(x)$

Con las $g(x)$ anteriores y la condición inicial $x_0 = 1.5$, se obtienen los siguientes resultados:

$g(x)$	$g'(x^* = 2)$	¿Converge?	Razón
$g_1(x) = x^2 - 2$	$g'_1(2) = 4$	No*	$ g'(2) = 4 > 1$
$g_2(x) = \sqrt{x+2}$	$g'_2(2) = 0.25$	Sí	$ g'(2) = 0.25 < 1$
$g_3(x) = \frac{2}{x-1}$	$g'_3(2) = -2$	No*	$ g'(2) = 2 > 1$

La misma ecuación puede converger o diverger según la transformación

Analizar en Geogebra y Python. $g_1(x) = x^2 - 2$ con $x_0 = 1$

¿Cómo se podría transformar la ecuación $\cos(x) = 0$?



Ejemplo: Cálculo de raíz con aproximaciones sucesivas

Problema: Encontrar la raíz de la ecuación siguiente usando el método de aproximaciones sucesivas

$$e^{-x} - x = 0$$

Usar una tolerancia de 0.001 y un valor inicial $x_0 = 1$



Ejercicio: Cálculo de raíz con aproximaciones sucesivas

Problema: Encontrar la raíz de la ecuación siguiente usando el método de aproximaciones sucesivas

$$2\sqrt{x} - x = 0$$

Usar una tolerancia de 0.001 y un valor inicial $x_0 = 1$



Implementación 3: Método de aproximaciones sucesivas

Objetivo: Implementar y comprender el algoritmo del método de aproximaciones sucesivas para encontrar raíces de ecuaciones.

Actividades:

1 Análisis algorítmico:

- Escribir el pseudocódigo completo del método de aproximaciones sucesivas
- Realizar el diagrama de flujo correspondiente
- Especificar claramente: entradas, proceso, salidas

2 Implementación en Python:

- Programar la función: `punto_fijo(g, x0, tol, max_iter)`
- La función debe realizar las verificaciones adecuadas en los datos de entrada y contar con criterio de detención según la tolerancia especificada y el número máximo de iteraciones.
- Se espera a la salida la raíz aproximada, el número de iteraciones y el error absoluto final.

Entregables (por Classroom):

- Documento PDF con: pseudocódigo, diagrama de flujo, código Python
- Archivo .py con la implementación

Fecha límite: 24 de marzo, 2026.



Tarea 5: Cálculo de raíz con aproximaciones sucesivas

Problema: Encontrar la raíz de la ecuación siguiente usando el método de aproximaciones sucesivas

$$\cos(x) = 0$$

Usar una tolerancia de 0.001 y un valor inicial $x_0 = 0.8$



Método de Newton-Raphson: Introducción

Similar a lo ocurrido con los métodos cerrados, en los abiertos es posible definir un algoritmo general cuyo cambio radica en la manera de calcular la siguiente aproximación. El método de Newton-Raphson es probablemente el más utilizado para el cálculo de raíces, pues suele ser **más eficiente** que los estudiados antes.



Figura: Isaac Newton (1643-1727)



Figura: Joseph Raphson (1648-1715)



Método de Newton-Raphson: Interpretación geométrica

El método consiste en trazar rectas tangentes a la función a partir del punto $[x_0, f(x_0)]$, el lugar donde la recta tangente cruza el *eje x*, representa una aproximación mejorada que se usa para trazar la siguiente tangente; esto se repite hasta obtener una aproximación que cumpla con la tolerancia.

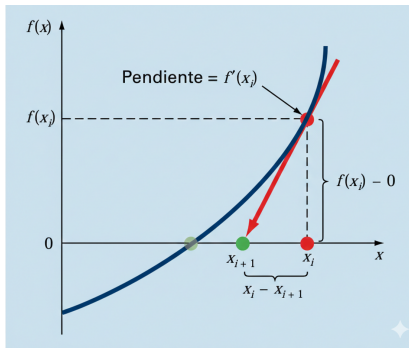
Fórmula de Newton-Raphson

La ecuación de la primera derivada de $f(x)$ respecto a x es la siguiente

$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}}$$

Reordenando la ecuación, se obtiene la *fórmula de Newton-Raphson*

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (1)$$



Ejemplo: Cálculo de raíz con Newton-Raphson

Problema: Usando el método de Newton-Raphson, calcular la raíz de la ecuación siguiente

$$e^{-x} - x = 0$$

Usar una tolerancia de 0.001 y un valor inicial $x_0 = -5$

Comparar el número de iteraciones necesarias para este mismo problema con el método de punto fijo.



Ejercicio: Cálculo de raíz con Newton-Raphson

Problema: Usando el método de Newton-Raphson, calcular una de las raíces positivas de la ecuación siguiente

$$\cos(x) = 0$$

Usar una tolerancia de 10^{-12} y un valor inicial $x_0 = 0.5$



¿Qué pasa si no puedo obtener la derivada?

Una complicación del método de Newton-Raphson radica en que su fórmula requiere la derivada de la función, pero hay casos en los que calcular dicha derivada puede resultar muy complejo.

Para resolver esto, se aproxima la derivada con una diferencia finita *hacia atrás*

$$f'(x_i) \approx \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

Al sustituir esta aproximación en la fórmula de Newton-Raphson, se obtiene el **método de la secante**.

Fórmula del método de la secante

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

Es similar a la ecuación de interpolación lineal del método de regla falsa; requiere 2 CI (x_i, x_{i-1}) .



Ejemplo: Cálculo de raíz con secante

Problema: Usando el método de la secante, calcular la raíz de la ecuación siguiente

$$\cos(x) = 0$$

Usar una tolerancia de 10^{-12} y valores iniciales $x_0 = 1$ y $x_1 = 2$.

Comparar el número de iteraciones necesarias para este mismo problema con el método de Newton-Raphson.



Implementación 4: Método de la secante

Objetivo: Implementar y comprender el algoritmo del método de la secante para encontrar raíces de ecuaciones.

Actividades:

1 Análisis algorítmico:

- Escribir el pseudocódigo completo del método de la secante
- Realizar el diagrama de flujo correspondiente
- Especificar claramente: entradas, proceso, salidas

2 Implementación en Python:

- Programar la función: `secante(f, x0, x1, tol, max_iter)`
- La función debe realizar las verificaciones adecuadas en los datos de entrada y contar con criterio de detención según la tolerancia especificada y el número máximo de iteraciones.
- Se espera a la salida la raíz aproximada, el número de iteraciones y el error absoluto final.

Entregables (por Classroom):

- Documento PDF con: pseudocódigo, diagrama de flujo, código Python
- Archivo .py con la implementación

Fecha límite: [Indicar fecha]



Tarea 6: Cálculo de raíz con secante

Problema: Usando el método de la secante, calcular la raíz de la ecuación siguiente

$$e^{-x} - x = 0$$

Usar una tolerancia de 10^{-12} y valores iniciales $x_0 = 0.5$ y $x_1 = 1$.

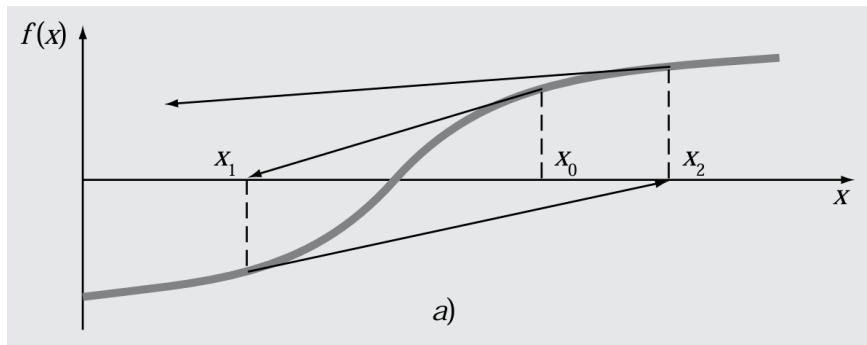
Comparar el número de iteraciones necesarias para este mismo problema con el método de Newton-Raphson.



Desventajas del método: Caso A

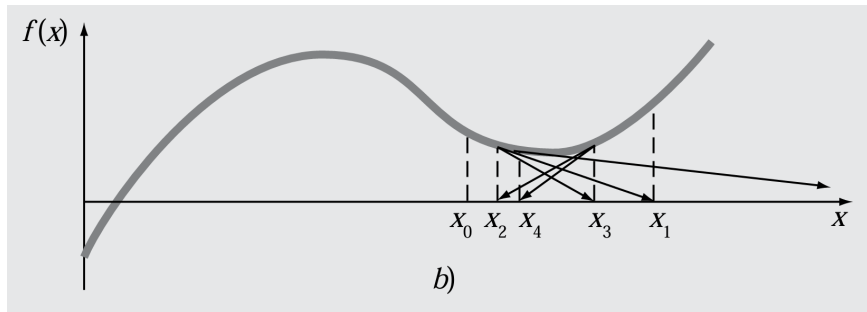
A pesar de la alta velocidad de convergencia, el método de Newton-Raphson presenta algunas desventajas, pues la naturaleza de la función determinará convergencia y su rapidez.

a) Cuando la raíz se encuentra en la vecindad de un punto de inflexión ($f''(x) = 0$), las iteraciones divergen progresivamente de la raíz.



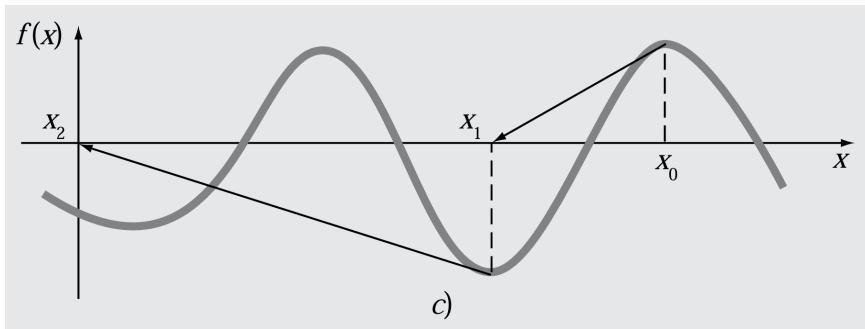
Desventajas del método: Caso B

b) El método tiende a oscilar en mínimos y máximos locales; estas oscilaciones pueden persistir indefinidamente o alcanzar una posición cerca al mínimo y provocar cambios abruptos en las iteraciones.



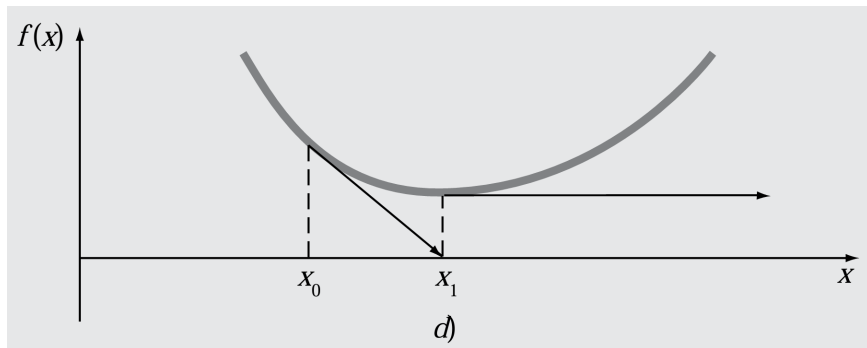
Desventajas del método: Caso C

c) Un valor inicial cercano a una raíz puede alejarse del área de interés, tendiendo a otras raíces, cuando se tienen pendientes cercanas a cero.



Desventajas del método: Caso D

d) El peor caso es cuando se llega un valor con pendiente cero, esto se traduce gráficamente en una recta paralela al eje x , es decir, no se puede obtener otra aproximación.



¿Qué ocurre aquí con la fórmula del método?

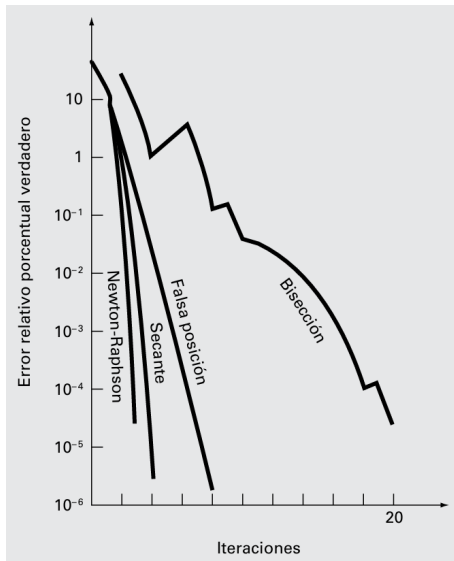


Conclusiones del método de Newton-Raphson

- No existe un criterio general de convergencia.
- La convergencia depende de la naturaleza de la función.
- Tener un valor inicial cercano a la raíz suele ayudar.
- Conocer el problema físico o apoyarse de recursos gráficos permite dar un valor inicial más cercano.
- Los programas para la implementación del método deben reconocer la convergencia lenta o la divergencia para ser eficientes.



Comparación de métodos vistos



¿Qué es?

Método iterativo para encontrar raíces de polinomios que extrae factores cuadráticos $x^2 + px + q$ de un polinomio de grado $n > 2$, permitiendo obtener tanto raíces reales como complejas.

También es conocido como **Método de Lin**
Aplicación:

- Polinomios de grado $n \geq 3$
- Coeficientes reales
- Búsqueda raíces reales y complejas

Ventajas:

- No requiere estimaciones iniciales
- Maneja raíces complejas
- Proceso sistemático e iterativo

L. Bairstow, 1880-1963



Fundamento del método: División polinomial

Polinomio original de grado n

$$P(x) = a_0x^n + a_1x^{n-1} + a_2x^{n-2} + \cdots + a_{n-1}x + a_n$$

División por un factor cuadrático

Se busca dividir $P(x)$ entre $x^2 + px + q$, de manera que

$$P(x) = (x^2 + px + q)(b_0x^{n-2} + b_1x^{n-3} + \cdots + b_{n-3}x + b_{n-2}) + Rx + S$$

Donde:

- $Rx + S$: Residuo lineal

Objetivo:

- Encontrar p y q tales que:

$$R \approx 0 \quad \text{y} \quad S \approx 0$$

Interpretación

Si $R = S = 0$, entonces $x^2 + px + q$ es factor exacto de $P(x)$, y sus raíces (reales o complejas) son raíces del polinomio original.



Fórmulas del método: b_k , R y S

Al multiplicar el factor cuadrático y el polinomio de grado $n - 2$, e igualar el resultado con el polinomio original, lo que se obtiene son las siguientes ecuaciones

$$b_k = a_k - pb_{k-1} - qb_{k-2}$$

$$R = a_{n-1} - pb_{n-2} - qb_{n-3}$$

$$S = a_n - qb_{n-2}$$

donde $k = 0, 1, 2, 3, \dots, n - 2$ y n es el grado del polinomio original. Considerando además que $b_{-1} = b_{-2} = 0$.



Fórmulas del método: p^* y q^*

Para un inicio, se considera $p = q = 0$, pero sus valores se actualizan desde la primera iteración con los siguientes incrementos

$$\Delta p = \frac{R}{b_{n-2}} \quad y \quad \Delta q = \frac{S}{b_{n-2}}$$

A partir de los valores anteriores de p y q y los incrementos calculados para cada uno, se calculan los nuevos valores de p y q que se usarán en cada iteración.

$$p^* = \Delta p + p$$

$$q^* = \Delta q + q$$

NOTA: $p = q = 0$ **NO** se usan en las iteraciones, sirven para calcular los primeros valores que sí se usan. En la primera iteración se usarán

$$p^* = \Delta p + 0 = \Delta p$$

$$q^* = \Delta q + 0 = \Delta q$$



Tabla de Iteraciones

La siguiente tabla organiza toda la información de cada iteración del método (con $p^{(0)} = 0$ y $q^{(0)} = 0$):

i	p^*	q^*	b_0	b_1	$\dots$	b_{n-2}	R	S	Δp	Δq
0	0	0	a_0	a_1	$\dots$	a_{n-2}	a_{n-1}	a_n	$\frac{a_{n-1}}{a_{n-2}}$	$\frac{a_n}{a_{n-2}}$
1	$p^{(1)}$	$q^{(1)}$	$b_0^{(1)}$	$b_1^{(1)}$	$\dots$	$b_{n-2}^{(1)}$	$R^{(1)}$	$S^{(1)}$	$\Delta p^{(1)}$	$\Delta q^{(1)}$
2	$p^{(2)}$	$q^{(2)}$	$b_0^{(2)}$	$b_1^{(2)}$	$\dots$	$b_{n-2}^{(2)}$	$R^{(2)}$	$S^{(2)}$	$\Delta p^{(2)}$	$\Delta q^{(2)}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	$p^{(k)}$	$q^{(k)}$	$b_0^{(k)}$	$b_1^{(k)}$	$\dots$	$b_{n-2}^{(k)}$	$R^{(k)}$	$S^{(k)}$	$\Delta p^{(k)}$	$\Delta q^{(k)}$

- i : Número de iteración
- p^*, q^* : Coeficientes actualizados del factor cuadrático
- b_k : Coeficientes del polinomio reducido
- R, S : Coeficientes del residuo
- $\Delta p, \Delta q$: Correcciones aplicadas



¿Cuándo detenerse?

En este método existen dos principales fuentes de información para generar un criterio de detención:

- **Residuo:** Si los coeficientes del residuo R y S son lo suficientemente pequeños, podemos considerar que el factor calculado es exacto.
- **Incremento del paso:** El cambio en los coeficientes del factor cuadrático Δp y Δq también sirve como medida del error, pues un salto muy pequeño, indica que la aproximación de la solución no podrá variar mucho más.

Aunque se puede seleccionar uno u otro para compararlo con la tolerancia y determinar si se debe detener el proceso, es necesario mencionar que el caso ideal es que ambos se cumplan al mismo tiempo.

NOTA: Recordar que siempre se debe añadir un número máximo de iteraciones aceptables como posible criterio de parada.



Ejemplo: Raíz con factores cuadráticos

Problema: Usando el método de los factores cuadráticos, encontrar las raíces del siguiente polinomio.

$$x^4 - x^3 + 6x^2 - 3x + 4 = 0$$

Considerar una tolerancia de 0.0001 en $R, S, \Delta p$ y Δq .



Ejercicio: Raíz con factores cuadráticos

Problema: Usando el método de los factores cuadráticos, encontrar las raíces del siguiente polinomio.

$$x^4 - 6x^2 - 6 = 0$$

Considerar una tolerancia de 0.0001 en $R, S, \Delta p$ y Δq .



Implementación 5: Método de Factores Cuadráticos (Lin)

Objetivo: Implementar y comprender el algoritmo del método de factores cuadráticos (método de Lin) para obtener raíces de polinomios de grado mayor o igual a tres.

Actividades:

1 Análisis algorítmico:

- Escribir el pseudocódigo completo del algoritmo para extraer un factor cuadrático.
- Realizar el diagrama de flujo correspondiente.
- Especificar claramente: entradas, proceso y salidas del algoritmo.

2 Implementación en Python:

- Programar la función: `factores_cuadraticos(coeficientes, tol, max_iter)`
- El algoritmo debe implementar el esquema iterativo que calcula los coeficientes intermedios b_k y actualiza los parámetros p y q .
- La función debe verificar las condiciones de convergencia usando una tolerancia especificada.
- Se espera a la salida los valores aproximados de p y q que definen el factor cuadrático $x^2 + px + q$ y el número de iteraciones realizadas.

Entregables (por Classroom):

- Documento PDF con: pseudocódigo, diagrama de flujo y código Python.
- Archivo .py con la implementación del algoritmo.

Fecha límite: 26 de marzo, 2026.



Desventajas del método: Polinomio incompleto

El método comienza con las propuestas de $p = q = 0$, a partir de las cuales se calculan los demás valores, sin embargo, analicemos **¿qué ocurre cuando un polinomio no tiene términos de todos los grados?**

Analicemos por ejemplo el polinomio $P(x) = x^4 - x - 2$, este es de 4° grado, es decir, $n = 4$ y $k = 2$; los coeficientes del polinomio original son

$$a_0 = 1, \quad a_1 = 0, \quad a_2 = 0, \quad a_3 = -1, \quad a_4 = -2$$

Lo que quiere decir que, al hacer el cálculo de los primeros incrementos, se tendrá una **división entre cero**

$$\Delta p = \frac{a_{n-1}}{a_{n-2}} = \frac{-1}{0}, \quad \Delta q = \frac{a_n}{a_{n-2}} = \frac{-2}{0}$$

Ya que estas ecuaciones provienen de considerar $p^{(0)} = q^{(0)} = 0$, se recomienda seleccionar otros valores iniciales **pequeños y distintos de cero**. Esto no garantiza el funcionamiento.



Eduardo Flores Rivas
Ingeniero Mecatrónico
Facultad de Ingeniería, UNAM
eduardo.flores@ingenieria.unam.edu

Bibliografía obligatoria y recomendada

 BURDEN, Richard L., FAIRES, J. Douglas
Análisis numérico.
9a. edición. México. Cengage Learning, 2011.

 CHAPRA, Steven C., CANALE, Raymond P.
Métodos numéricos para ingenieros.
6a. edición. México. McGraw-Hill, 2011.

 GERALD, Curtis F., WHEATLEY, Patrick O.
Análisis numérico con aplicaciones.
6a. edición. México. Prentice Hall / Pearson Educación, 2000.

